

Documentación: Especificación de la Arquitectura

Brayan Esteban Alpízar Elizondo
Mariana González Sanabria
Frederick Obando Solano
Diego Salas Ovarés

Compiladores e Interpretes
Instituto Tecnológico de Costa Rica

4 de mayo de 2026

1. Tamaño de Palabra

El ISA propuesto utiliza un tamaño de palabra de **32 bits** tanto para datos como para instrucciones. Esto implica que cada instrucción ocupa exactamente una palabra de memoria, lo cual es consistente con arquitecturas tipo RISC, permitiendo un diseño de hardware más simple y eficiente.

1.1. Codificación de instrucciones

Las instrucciones se codifican dentro de esta palabra de 32 bits mediante diferentes **formatos de instrucción**, dependiendo del tipo de operación. Todos los formatos comparten ciertos campos comunes, como:

- ✓ **P (bit 0)**: bit de seguridad.
- ✓ **opcode (bits 1-5)**: identifica el tipo de instrucción.

A partir de estos campos, el resto de los bits se interpretan según el formato de la instrucción:

Tipo R (registro-registro)

- ✓ **func7 (bits 25-31)**
- ✓ **rm (bits 20-24)**
- ✓ **rn (bits 15-19)**
- ✓ **rd (bits 10-14)**
- ✓ **func4 (bits 6-9)**

Tipo I (registro-inmediato)

- ✓ imm12 (bits 20-31)
- ✓ rn (bits 15-19)
- ✓ rd (bits 10-14)
- ✓ func4 (bits 6-9)

Tipo M (memoria)

- ✓ imm12 (bits 20-31)
- ✓ rn (bits 15-19)
- ✓ rd (bits 10-14)
- ✓ bits de control:
 - S (bit 6): suma/resta
 - B (bit 7): acceso a byte
 - H (bit 8): acceso a media palabra
 - W (bit 9): acceso a palabra

Tipo B (branch)

- ✓ imm12 dividido en:
 - imm12[11:5] (bits 25-31)
 - imm12[4:0] (bits 10-14)
- ✓ rm (bits 20-24)
- ✓ rn (bits 15-19)
- ✓ cond (bits 6-9)

Tipo J/F (saltos)

- ✓ imm21 dividido en:
 - imm21[20:4] (bits 15-31)
 - imm21[3:0] (bits 6-9)
- ✓ rd (bits 10-14)

Tipo S (sesión de seguridad)

- ✓ imm20 (bits 12-31)
- ✓ func4 (bits 6-9)

Tipo V (bóveda)

- ✓ imm16 (bits 16-31)
- ✓ sn (bits 13-15)
- ✓ sd (bits 10-12)
- ✓ bits de control S, B, H, W (bits 6-9)

Tipo PR (procesamiento seguro)

- ✓ func7 (bits 25-31)
- ✓ func3 (bits 22-24)
- ✓ sf (bits 19-21)
- ✓ sm (bits 16-18)
- ✓ sn (bits 13-15)
- ✓ sd (bits 10-12)
- ✓ func4 (bits 6-9)

Tipo PI (procesamiento seguro inmediato)

- ✓ imm16 (bits 16-31)
- ✓ sn (bits 13-15)
- ✓ sd (bits 10-12)
- ✓ func4 (bits 6-9)

Tipo T (transferencia)

✓ send:

- rn (bits 15-19)
- sd (bits 10-12)

✓ recv:

- sm (bits 16-18)
- rd (bits 10-14)

✓ func4 (bits 6-9)

La codificación sigue un diseño modular donde el **opcode** define el tipo de instrucción y los campos restantes se reutilizan según el formato. Esto permite:

- ✓ Simplicidad en el decodificador
- ✓ Extensibilidad (ej. extensión de seguridad)
- ✓ Compatibilidad con pipeline

2. Conjunto de Registros

El ISA define dos bancos de registros: un banco principal de propósito general y un banco seguro utilizado por la extensión de seguridad.

2.1. Banco de registros principal

El banco principal contiene **32 registros**, cada uno identificado mediante una dirección de **5 bits**. Estos registros se utilizan para operaciones aritméticas, lógicas, control de flujo, manejo del stack, llamadas a funciones y almacenamiento temporal.

Tabla 1. Banco principal de registros del ISA.

Código	Registro	Propósito
0	zero	Constante cero, solo lectura
1	ra	Dirección de retorno de funciones
2	sp	Puntero de pila
3	pc	Contador de programa
4	lr	Registro de sesión segura, solo lectura
5	p0	Argumento y valor de retorno de función
6–13	p1--p8	Argumentos de función
14–29	r0--r15	Registros de propósito general y temporales
30	delta	Constante 0x9e3779b9, solo lectura
31	max	Constante 0xFFFFFFFF, solo lectura

2.2. Uso de registros

Según la convención del ISA, los registros se utilizan de la siguiente forma:

- ✓ **Parámetros de función:** se usan los registros `p0--p8`.
- ✓ **Valor de retorno:** se usa el registro `p0`.
- ✓ **Almacenamiento temporal:** se usan los registros `r0--r15`.
- ✓ **Dirección de retorno:** se usa el registro `ra`.
- ✓ **Puntero de pila:** se usa el registro `sp`.
- ✓ **Contador de programa:** se usa el registro `pc`.
- ✓ **Estado de sesión segura:** se usa el registro `lr`.

2.3. Banco seguro de registros

Además del banco principal, el ISA define un banco seguro compuesto por **8 registros**, identificados mediante direcciones de **3 bits**. Estos registros son utilizados por las instrucciones de la extensión de seguridad.

Tabla 2. Banco seguro de registros del ISA.

Código	Registro	Propósito
0	<code>ax</code>	Propósito general seguro
1	<code>bx</code>	Propósito general seguro
2	<code>cx</code>	Propósito general seguro
3	<code>dx</code>	Propósito general seguro
4	<code>ex</code>	Propósito general seguro
5	<code>fx</code>	Propósito general seguro
6	<code>gx</code>	Propósito general seguro
7	<code>hx</code>	Propósito general seguro

El conjunto de registros del ISA está organizado para separar las operaciones generales de las operaciones seguras. El banco principal permite manejar funciones, temporales, flujo de control y constantes especiales, mientras que el banco seguro permite ejecutar operaciones protegidas mediante la extensión de seguridad.

3. Organización de Memoria

La arquitectura objetivo del compilador utiliza una memoria direccionable total de **64 KB**, es decir:

$$64 \text{ KB} = 65\,536 \text{ bytes} = 0x10000 \text{ bytes}$$

Por tanto, todo el programa compilado debe caber dentro de ese espacio, incluyendo código, datos globales y memoria de pila.

Organización general

La memoria se organiza en los siguientes segmentos:

- ✓ **Segmento de código:** contiene las instrucciones ensamblador generadas por el compilador.
- ✓ **Segmento de datos globales:** contiene variables globales y demás datos estáticos del programa.
- ✓ **Segmento de pila (stack):** contiene variables locales, arreglos locales, parámetros que deban guardarse temporalmente y datos de activación de funciones.
- ✓ **Heap:** en la versión actual del compilador **no se implementa heap** ni reserva dinámica de memoria.

Distribución de memoria

La distribución real se calcula al finalizar la generación de ensamblador:

1. Primero se genera todo el **código**.
2. Luego, el segmento de **datos globales** se ubica inmediatamente después del código, alineado a palabra.
3. Finalmente, la base de la **pila** se coloca después de los datos globales, también alineada.

De esta manera se evita traslape entre código, datos y pila. Además, el compilador valida que el tamaño total utilizado no exceda los 65536 bytes disponibles.

Asignación de direcciones

Variables globales Las variables globales reciben una dirección absoluta dentro del segmento de datos. Su dirección final no se fija de forma definitiva al inicio, sino que se reajusta una vez conocido el tamaño final del código generado.

Funciones Cada función recibe una dirección dentro del segmento de código. Esa dirección corresponde a la posición de su etiqueta de entrada en el ensamblador generado.

Variables locales Las variables locales se almacenan dentro del **frame** de activación de su función. El compilador reserva espacio en la pila para:

- ✓ dirección de retorno,
- ✓ variables locales escalares,
- ✓ arreglos locales,
- ✓ copias temporales de parámetros cuando se requiere.

Cada variable local recibe un desplazamiento dentro de ese frame.

Parámetros Los primeros parámetros de una función se pasan mediante registros de parámetro (`p0`, `p1`, ...). Si un parámetro necesita preservarse en memoria o tratarse como base direccionable, el compilador le reserva espacio adicional dentro del frame.

Manejo del espacio

El compilador calcula el tamaño de cada objeto según su tipo:

- ✓ `int`, `float`, `bool`: 4 bytes
- ✓ `char`: 1 byte
- ✓ punteros: 4 bytes
- ✓ arreglos: tamaño del elemento multiplicado por la cantidad de elementos
- ✓ `vault[n]`: se reserva como un bloque contiguo de `n` palabras, es decir, `4n` bytes

Todos los espacios de memoria relevantes se alinean a palabra cuando corresponde, con el fin de mantener consistencia con la ISA y simplificar los accesos.

Gestión de pila

La pila se usa únicamente para almacenamiento temporal de ejecución. En esta implementación:

- ✓ al entrar a una función, se reserva su frame en pila;
- ✓ al salir, ese espacio se libera completamente;
- ✓ la pila se utiliza para variables locales, arreglos locales y soporte de llamadas.

Además, el compilador verifica que el frame más grande posible, junto con código y datos globales, siga cabiendo dentro de la memoria total de 64 KB.

En la versión desarrollada, la memoria queda organizada de forma **estática y controlada**:

- ✓ código al inicio,
- ✓ datos globales a continuación,
- ✓ pila después de los datos,
- ✓ sin heap dinámico.

Esta decisión simplifica el diseño del compilador y permite validar de manera temprana que el programa generado respete los límites de memoria de la arquitectura propuesta.

4. Conjunto de Instrucciones

El ISA define un conjunto amplio de instrucciones que cubren operaciones aritméticas, lógicas, acceso a memoria, control de flujo y operaciones de seguridad. Estas instrucciones se agrupan según su tipo y funcionalidad.

4.1. Clasificación de instrucciones

✓ Carga y almacenamiento:

- ldw, ldh, ldb
- stw, sth, stb

✓ Aritméticas:

- add, sub, mul, div, mod
- addi, subi, muli, divi, modi

✓ Lógicas:

- and, orr, xor
- andi, orri, xori

✓ Comparación:

- seq, seqi

✓ Desplazamiento:

- sll, srl
- slli, srli

✓ Saltos:

- beq, bne, bgt, blt, bge, ble

- jal, jmp

✓ **Control de flujo:**

- call, ret

✓ **Especiales:**

- nop

✓ **Seguridad:**

- login, quit

✓ **Extensión segura (registros seguros):**

- Acceso a bóveda: ldvw, stvw, ldvh, stvh, ldvb, stvb
- Aritméticas seguras: padd, psub, pmul, pdiv, pmod
- Lógicas seguras: pand, porr, pxor
- Extendidas: paddadd, pxorxor, pslladd, psrladd
- Inmediatas: paddi, psubi, pmuli, pdivi, pmodi

✓ **Transferencia:**

- send, recv

4.2. Opcode y formato

Cada instrucción se identifica mediante un **opcode de 5 bits**:

Tabla 3. Codificación de opcodes.

Opcode	Tipo
00000	R
00001	I
00010	PR
00011	PI
00100	M (load)
00101	M (store)
00110	V (load)
00111	V (store)
01000	B
01001	J/F
10000	T
10001	S

4.3. Formato de instrucciones

Las instrucciones utilizan distintos formatos según el número de operandos:

- ✓ **R**: tres registros (**rd**, **rn**, **rm**)
- ✓ **I**: dos registros + inmediato (**rd**, **rn**, **imm**)
- ✓ **M**: acceso memoria (**rd**, $\pm$ **imm**(**rn**))
- ✓ **B**: comparación + salto (**rn**, **rm**, **imm**)
- ✓ **J**: salto (**rd**, **imm**)
- ✓ **PR**: registros seguros (**sd**, **sn**, **sm**)
- ✓ **PI**: registros seguros + inmediato (**sd**, **sn**, **imm**)
- ✓ **T**: transferencia (**send sd**, **rn** / **recv rd**, **sm**)

4.4. Ejemplos de uso

```
add r1, r2, r3      // suma
addi r1, r2, 10     // suma con inmediato
ldw r1, 0(r2)       // carga de memoria
stw r1, 4(r2)       // almacenamiento
beq r1, r2, label   // salto condicional
jal ra, func        // llamada a funcin
ret                // retorno

login 0xBEEF0       // inicio de sesin segura
padd ax, bx, cx     // operacin segura
send ax, r1         // transferencia a registro seguro
```

El conjunto de instrucciones es completo y extensible, permitiendo operaciones generales y seguras. La organización por tipos y el uso de opcodes definidos facilita la implementación del decodificador y el soporte para ejecución en pipeline.